Journal of Computer Languages 86 (2026) 101386

Contents lists available at ScienceDirect

Journal of Computer Languages

journal homepage: www.elsevier.com/locate/cola

# The Stackage repository: An exploratory study of its evolution☆

Paul Leger [a,*], Felipe Ruiz [b], David Moreno-Lumbreras [c], Nicolás Sepúlveda [b], Ismael Figueroa [b], Nicolás Cardozo [d]

[a] Escuela de Ingeniería, Universidad Católica del Norte, Coquimbo, Chile
[b] Pragmatics Lab, Chile
[c] Escuela de Ingeniería de Fuenlabrada, Universidad Rey Juan Carlos, Fuenlabrada, Spain
[d] Systems and Computing Engineering Department, Universidad de los Andes, Bogota, Colombia

ARTICLE INFO

*Keywords:*
Stackage
Haskell
Mining software repository

ABSTRACT

Curated software ecosystems aim to ensure stability and reproducibility, yet their long-term evolution remains poorly understood. This paper presents an empirical study of Stackage, the curated repository of Haskell packages, analyzing 35 releases from 2014 to December 2025 (93,377 packages), more than a decade. By combining quantitative and qualitative analyses of dependency networks, update activity, and monad abstraction usage, we characterize how curation shapes ecosystem evolution. Our results reveal that Stackage evolves through controlled growth: dependency structures remain stable, but a small fraction of highly connected packages accumulate instability risk. The dominant monad libraries (mtl/transformers) exhibit long-term semantic stability.

## 1. Introduction

Modern software development increasingly depends on large ecosystems of reusable components distributed through online repositories. While these ecosystems accelerate innovation and reuse, they also introduce complex webs of interdependencies that challenge maintainability, stability, and long-term maintainability [1–3]. Understanding how such ecosystems evolve, stabilize, and consolidate their abstractions has therefore become a central concern in empirical software engineering.

Among these ecosystems, the Haskell community provides a unique and insightful contrast between two models of evolution: Hackage, an open repository where packages are freely published, and Stackage, a curated subset of Hackage packages that ensures build stability and compatibility across versions. Unlike uncurated ecosystems such as nom or PyPI, where dependency networks evolve organically and often unpredictably, Stackage enforces compatibility and controlled growth through its Long-Term Support (LTS) releases. This makes it an ideal setting to explore how *curation shapes software evolution* balancing stability, reuse, and innovation.

Despite its relevance, the longitudinal evolution of Stackage remains largely unexplored. Previous research has focused mainly on Haskell abstractions in Hackage [4–6], but little is known about how curated repositories evolve internally: how dependencies change, how core packages consolidate their role, and how abstraction layers such as monads persist or adapt over time. This study addresses that gap through a mixed quantitative and qualitative exploration of Stackage's evolution, combining dependency-level metrics with semantic analysis of widely used monad packages.

Stackage is widely used in the Haskell community, with over 85 million package downloads and 35 Long-Term Support (LTS) releases managed between December 2014 and December 2025. Its curated nature allows developers to build software on a consistent dependency base. However, this same stability raises an empirical question: *how does a curated ecosystem evolve without compromising its promise of stability?* Our goal is to uncover the mechanisms through which curation influences dependency health, maintenance activity, and abstraction use across nearly a decade of releases.

To address this goal, we analyze the Stackage repository by parsing its package manifests (.cabal files) using the *Cabal* system [7] and custom Python and Haskell scripts that extract dependency information, versioning data, and module imports from source code. We collected and analyzed data from 35 LTS releases, comprising 93,377 packages

☆ This paper extends a study presented on ACM Symposium on Applied Computing (Leger et al., 2022).

* Corresponding author.
*E-mail addresses:* pleger@ucn.cl (P. Leger), felipe.ruiz@alumnos.ucn.cl (F. Ruiz), david.morenolu@urjc.es (D. Moreno-Lumbreras), nicolas.sepulveda@alumnos.ucn.cl (N. Sepúlveda), ifigueroap@gmail.com (I. Figueroa), n.cardozo@uniandes.edu.co (N. Cardozo).
*URL:* http://pragmaticslab.com (P. Leger).

https://doi.org/10.1016/j.cola.2026.101386
Received 24 October 2025; Received in revised form 24 January 2026; Accepted 27 January 2026
Available online 6 February 2026
2590-1184/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(32.04 GB of source code), to examine both structural and semantic aspects of evolution.

Our findings provide new insights for both researchers and practitioners. From a research perspective, the results contribute to understanding how curation affects dependency networks and abstraction longevity, offering evidence that curated ecosystems evolve through deliberate constraint rather than random change. From a practical standpoint, the study offers guidance for maintainers of Stackage and similar ecosystems, highlighting how stability mechanisms, dependency validation, and abstraction reuse can be balanced to ensure sustainable growth.

Concretely, this paper analyzes and characterizes how Stackage grows across releases while maintaining a stable core of widely used packages, and how changes in dependency constraints and revisions of package manifests relate to compatibility issues observed at release time. We also identify highly connected packages and long dependency chains as cases where small instabilities can have disproportionate downstream impact. These observations translate into actionable signals that maintainers can use to prioritize review and governance efforts in curated releases.

This paper extends our previous work presented at the ACM Symposium on Applied Computing (SAC 2022) [8] in several significant ways:

1. **Expanded dataset**: from 14 to 35 LTS releases (a 150% increase), covering more than a decade of repository evolution.
2. **Enhanced research scope**: refined and added RQs to better capture structural and semantic evolution (notably a new RQ4 and revised RQs 5–6).
3. **Broader analysis**: inclusion of additional monad packages (`mtl`, `transformers`, `monad-control`, and `free`) and deeper investigation of their usage trends.
4. **Qualitative enrichment**: manual inspection of core packages and inclusion of insights shared by the Stackage maintainers via a public GitHub discussion.
5. **Expanded theoretical framing**: stronger connection to research on software dependencies, maintenance activity, and curated ecosystem governance.
6. **New *Discussion* section**: rewritten to integrate all RQs, interpret the findings conceptually, and highlight the role of curation in software maintainability.
7. **Automatized scripts**: A set of Python scripts to download and process the Stackage releases. Using Jupyter,[1] a web-based interactive computing platform for different programming languages. The scripts and data used are available on https://github.com/pragmaticslaboratory/stackage-evolution (rev. `7f88d95`) [9].

The remainder of this paper is structured as follows. Section 2 introduces background concepts on software repository evolution and monads, and Section 3 provides an overview of the Stackage repository as our curated case study. Section 4 then details our data collection and analysis methodology, and Section 5 addresses the research questions. Section 7 discusses related work, and Section 8 concludes with the main lessons learned and future research directions.

## 2. Evolution of package repository and monads

This section presents the core concepts needed to understand this study. First, we introduce concepts of package repositories, Mining Software Repositories (MSR) [10], and repository evolution. Second, we introduce monads, the main packages offering monad functionality, and their uses. In the next section, we build on this background by focusing on Stackage as our curated Haskell repository case study.

[1] http://jupyter.org

### 2.1. Package repository evolution

Package repositories allow developers to store, access, and share packages (*e.g.,* GitHub[2]). Package repositories can be used and constrained to different domains, like Linux distributions (*e.g.,* Debian[3]), or software development (*e.g.,* GitLab[4]). Of interest, as a subject to our study, are programming language specific repositories. Using such repositories developers create a community around a programming language, to create, access, and share their packages. Examples of specific repositories are npm [11] for JavaScript, PyPI [12] for Python, NuGet [13] for .Net, and Hackage [14] for Haskell.

A repository can evolve issuing different releases over time, where each release contains a list of selected package versions that do not, ideally, have conflicting dependencies among them. In Haskell, developers can use Stackage [15], a repository of curated Haskell packages that are selected from Hackage, as Stackage keeps a set of LTS releases since 2014, with 35 LTS releases available as of December 2025, comprising the 93,377 packages analyzed in this study.

MSR focuses on the empirical analysis of the data available in software repositories. Empirical studies have allowed developers and researchers to gather insight and build a panorama of a repository and its packages [16–21]. For such studies, different metrics (*e.g.,* repositories' health [22]) and visualization techniques are used to analyze the repositories [23,24]. Additionally, different tools like G-Repo [25] (a tool to analyze GitHub projects) or BOA [26] (a Domain Specific Language (DSL) to query data related to repository software) are used to analyze the evolution and distribution of software repositories in detail [27].

Table 1 lists MSR studies, ordered by year, related to the use of packages. These studies focus on two aspects. First, *exploring* a package repository to characterize it (*e.g.,* most used packages, number of packages, package update frequency). Second, analyzing *dependencies* and *vulnerabilities* that contain the packages in a (release of the) repository. The majority of studies focus on JavaScript and R repositories. Kikas et al. [19] analyze the dependency network of software repositories for JavaScript, Ruby, and Rust. Plakidas et al. [21] conduct a study on the evolution of a repository for the R programming language using package's metadata. Similarly, there is an analysis identifying the *technical debt* among package dependencies (*i.e.,* outdated dependencies) [17,28]. Decan et al. [29] analyze seven repositories (NPM, CRAN R, RubyGems, CARGO, CPAN, NuGet, and Packgist) to find similarities and differences between the evolution of package dependency networks for these repositories, which contain varying sizes and ages. Finally, Hu et al. [18] explore the *influence* of GitHub repositories from their users (developers) between 2012 and 2015, where the influence is measured by the number of *forks* and *stars* that a repository contains.

### 2.2. Monads in haskell

The Haskell programming language has been relevant to progress in the robust software development; indeed, the Haskell symposiums within International Conference on Functional Programming (ICFP) is annually hosted.[5] The rationale concepts behind Haskell are relevant for *critical systems* to guarantee their reliability, where an error can imply financial losses or irreversible security compromises. For example, robust software engineering through system types (*e.g., Liquid Haskell* [43]) or blockchain through Cardano [44] with *EUTxO (Extended Unspent Transaction Output)* to develop decentralized application (DApp) using *Smart Contracts* [45], a transaction protocol that is intended to automatically execute, control, or document events and

[2] http://github.com
[3] http://wiki.debian.org/DebianReleases
[4] http://docs.gitlab.com/ee/user/project/releases
[5] https://icfp24.sigplan.org. ICORE Conference Portal Rank: A.

**Table 1**
MSR studies on package use for programming languages, ordered by year.

| Year | Repository system | Language(s) | Objective | Summary |
|---|---|---|---|---|
| 2008 [30] | CVS | C, C++, Java (mainly) | Exploratory | Analysis of the evolution of software using CVS/Subversion between source code and other kinds of files (*e.g.*, HTML, XML). |
| 2010 [6] | Hackage | Haskell | Exploratory | Analysis in Hackage to find out design patterns. |
| 2011 [31] | Git | Ruby | Exploratory | Overview of the open source of ecosystem Ruby. |
| 2013 [32] | Hackage | Haskell | Exploratory | Analysis in Hackage to know how developers use generic programming language constructs, tools, and libraries in Haskell. |
| 2013 [33] | GNU R | R | Exploratory | Analysis of the ecosystem of R packages. |
| 2016 [34] | npm | JavaScript | Exploratory | Analysis of the npm ecosystem and its evolution. |
| 2016 [35] | npm, CRAN R, PyPI | JavaScript, R, Python | Dependencies | Analysis of npm, PyPI, and CRAN R to find differences between these ecosystems. |
| 2016 [36] | CRAN R | R | Exploratory | Analysis of dependency constraints and package versions in R. |
| 2017 [19] | GitHub | JavaScript, Ruby, Rust | Dependencies | Analysis of how updating or eliminating dependencies affects a repository that strongly depends on specific packages, giving rise to potential errors in the repository. |
| 2017 [21] | CRAN R | R | Exploratory and metrics | Descriptive study of packages in the CRAN repository of the R programming language to give a vision of the distribution and identity that R has for the developer community. |
| 2017 [37] | npm, CRAN R, RubyGems | JavaScript, R, Ruby | Dependencies | Analysis of dependency version specification, pointing out that the current tools and versioning schemes can introduce resiliency issues to the ecosystem |
| 2018 [29] | npm, CRAN R, RubyGems, CARGO, CPAN, NuGet, Packgist (Libraries.io) | JavaScript, R, Ruby, Rust, Perl, .NET languages, PHP | Dependencies | Analysis of the similarities and differences between the evolution of package dependency networks for seven packaging ecosystems of varying sizes and ages. |
| 2019 [38] | PyPI | Python | Exploratory | Descriptive study of Python packages to classify dependencies, licenses, imports, releases |
| 2018 [1] | npm | JavaScript | Dependencies | Analysis to identify the *technical lag* among npm package dependencies. |
| 2021 [39] | npm | JavaScript (NodeJs) | Dependencies | Analysis of the lag in packages with vulnerable dependencies. |
| 2021 [5] | Hackage | Haskell | Exploratory | Analysis of the mtl monads in Hackage. |
| 2022 [40] | npm, RubyGems | JavaScript, Ruby | Vulnerabilities | Analysis of the vulnerabilities in npm and RubyGems to know when and how they are fixed. |
| 2023 [41] | GitHub | TypeScript (typed JavaScript) | Exploration | Analysis of adoption of 13 new features over the last 3 years. |
| 2024 [42] | Packgist (Libraries.io) | Python | Dependencies | Analysis of dependencies and conflict issues over 278 (top popular) projects. |

actions according to the terms of a contract or an agreement in the context of a blockchain.

Monads are defined as abstractions that allow developers to encapsulate and reason about *notions of computations*, such as mutable state, I/O, or exceptions through a denotational approach. Monads were originally introduced in Haskell [46,47], and later extended to the concept of monad transformers [48]. Additionally, monads put forward theoretical foundations [49] that have even been applied in other domains such as Aspect-Oriented Programming [50], and more recently Context-Oriented Programming [51]. Furthermore, to simplify the use of monads, they have been introduced as a *design pattern* [52].

In the Haskell community, we can find a number of packages that provide monads with standard or custom implementation such as: transformers, the monad transformer library (*a.k.a.* mtl), monad-control, free, monads-tf, or monads-fd. In this study, we focus on the former four monad packages, which constitute the main packages used by the Haskell community.

The transformers package implements the functor and monad functionality as a portable library used to lift state transformers into higher-order functions that can be used generally by programs [53]. The mtl package builds on the ideas of transformers, providing a standardized and flexible use of monads [48] through introducing the Monad class. Using monad classes, it is possible to have more generic lifting of the monadic behavior through functional dependencies. The monad-control package is used to lift generic control operations existing in other monads (*e.g.,* IO). Finally, the free package offers unrestricted monads to use in managing tree-like structures or the implementation of DSLs.

There are different studies regarding the use of monads in Haskell (*cf.* Section 7). Algehed and Russo [54] show how monads can be used to track data dependencies statically. Benton et al. [55] explain the impact of monads in terms of applications in pure functional languages like Haskell. Unimo [56] is a framework that enforces upon developers the modular definition of monads. Yampa [57] brings Functional Reactive Programming [58] to Haskell through the arrows monads. Finally, the monatron monad [59] addresses flexibility and expressiveness issues in existing monads.

## 3. A brief overview of stackage

Fig. 1 shows the provenance of Stackage packages from the central package repository, Hackage; as a matter of fact, the Stackage name comes from "STAble haCKAGE". To clarify the relationship between Hackage and Stackage, Table 2 lists the main differences between both repositories. We briefly describe the Stackage repository.

Fig. 2 shows the release evolution during the analyzed time period, specifically showing the number of packages and Lines-Of-Code (LOC)

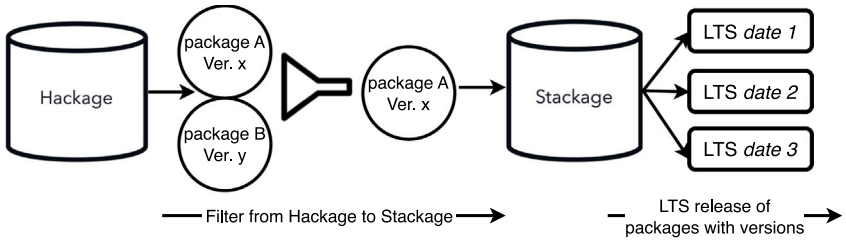



**Fig. 1.** Relationship between Hackage and Stackage.

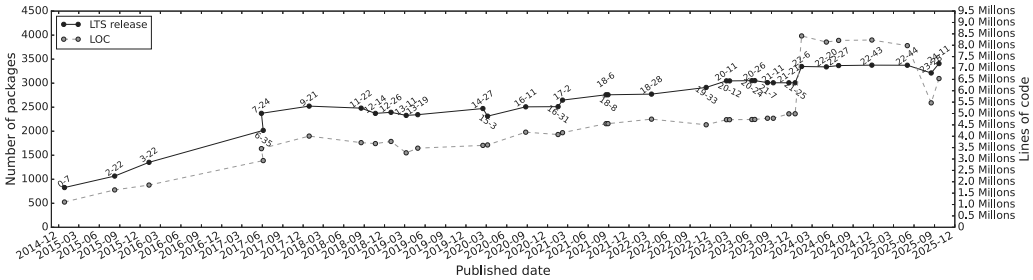



**Fig. 2.** Size and number of packages per Stackage release over time.

**Table 2**
Differences between Hackage and Stackage.

| Feature | Hackage | Stackage |
|---|---|---|
| Nature | Open Package repository | Curated and tested package sets |
| Stability | Can have dependency issues | Should ensure package compatibility |
| Update | Packages updated individually | Packages included in LTS (and Nightly snapshots) |
| Flexibility | More up-to-date but less stable | More stable but may have older versions |

per release, and the availability date for the releases. From the figure, we can observe that the number of packages and LOC increase proportionally, hence, we next refer to both as packages. In addition, we can see how Stackage had a gradual growth of packages throughout its first releases, highlighting that there are gaps of variating time spans (on average 6 months) between releases, until release 6–35, where the time between releases becomes more frequent and consistent. In the subsequent LTS releases up to December 2025, the total number of packages continues to grow more slowly, fluctuating within a narrower range than in the initial years. Finally, there are updates that are made in short intervals of time. For example, we can see that starting at release 11–22, there are 5 updates in approximately one year, and later releases preserve similarly short time spans between some consecutive LTS versions. It is relevant to note that these releases follow a fixed and non-uniform schedule.

***Amazon AWS in Stackage.*** We observed a net increase from LTS 21.25 (3008 packages) to LTS 22.6 (3347; +339), largely explained by the introduction of the AWS ecosystem: 288 new amazonka-* packages appear in LTS 22.6 (absent in LTS 21.25), alongside normal churn (531 added vs. 192 removed). Conversely, the net decrease from LTS 22.44 (3373) to LTS 23.28 (3212; -161) is mainly driven by the removal of the amazonka-* family during the transition to the newer LTS series (with a compiler/base upgrade), while the remainder continues typical turnover (290 added vs. 451 removed).

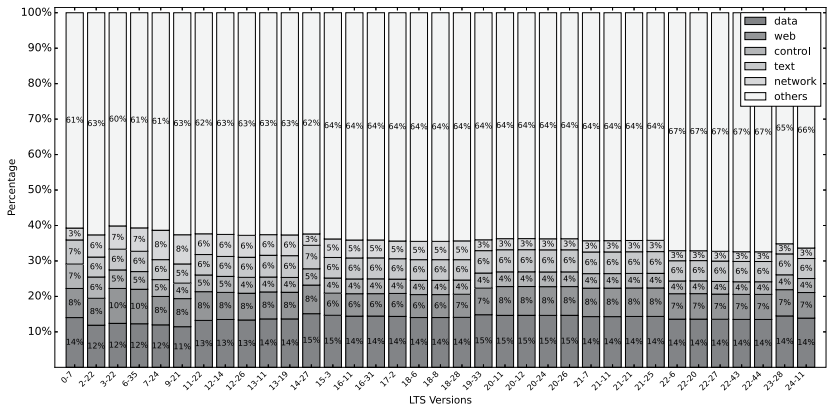
Fig. 3 describes the release evolution through categories and stability of packages. Fig. 3(a) presents the evolution of the top five package categories (*i.e.,* data, network, control, text, web) over different releases. From the figure, we can see that categories such as web and control show a slight decrease in their participation percentage compared to the earliest releases. Other categories, like data, despite varying over time, have a constant average percentage of their presence in Stackage. Finally, the network category shows moderate fluctuations over time, with local increases in intermediate releases but no monotonic growth when comparing the earliest and latest LTS releases. This suggests that while network-related packages remain consistently present, their relative proportion stabilizes rather than steadily increases over the long term. Fig. 3(b) presents the presence of stable packages over time. From the figure, we can observe that throughout its releases, the information in the stability field has decreased notably. The consequence of this is that most packages in Stackage do not have a defined type of stability. Therefore, most packages with information about stability are classified as experimental, and this pattern persists in the additional releases included in our updated dataset.

It is also worth noting that the others category consistently represents more than half of the packages across all releases. This dominance reflects the unstructured and developer-defined nature of the category field, which limits the granularity of category-based analyses. A finer-grained recategorization could further strengthen evolutionary insights, but is outside the scope of this study and left for future work.

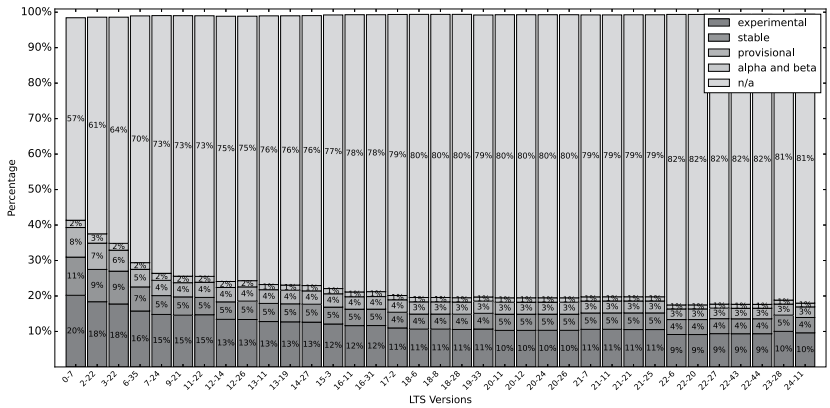
As mentioned in Section 4.2, the *category* and *stability* fields are strings without format. As a consequence, we can find many different categories and stabilities that are not significant to show in Fig. 3(b) (*e.g.,* empty string). These findings corroborate results from previous studies [4,5], which analyze the current Hackage repository. The reason for the poor results is mainly due to developers not following a protocol when submitting a new package release.

### 3.1. Curation process and package inclusion

A key difference between Hackage and Stackage is that Stackage snapshots are curated: packages are selected to build together under a consistent set of dependency constraints. In practice, this curation is managed through a transparent, community-facing workflow hosted in the official Stackage GitHub repository (https://github.com/




(a) Package percentage by category




(b) Package percentage by stability level

**Fig. 3.** Package percentage distribution with respect to category and stability.

commercialhaskell/stackage), in which maintainers and contributors propose and review snapshot changes via pull requests and automated checks.

At a high level, package inclusion is driven by edits to constraint files validated by CI. To propose adding a package to Stackage *Nightly*, contributors update the `build-constraints` file and open a pull request; to propose adding a package to Stackage *LTS*, contributors open a pull request against the `lts-haskell` repository, updating its `build-constraints`.

**Inclusion requirements (summary).** Stackage welcomes packages provided that:

- The package builds from its Hackage release in the current Stackage snapshots;
- Snapshot builds use *pristine sources* from Hackage (*i.e.,* no downstream patching is applied beyond what is available via Hackage revisions);
- The package is compatible with the versions of core libraries shipped with the target GHC version;
- Tests/benchmarks and CI are encouraged, and maintainers aim to follow the expectations documented in `MAINTAINERS.md`.

For full details on how to add and test a package, we refer readers to the official instructions in the Stackage repository.

### 3.2. Statistic analysis

To identify genuine variance over time, we apply the *One-Way Anova* test [60], given that our dataset does not follow a *normal distribution*. In this analysis, each release corresponds to one group, and the variables of interest are the per-release distributions derived from the metadata (*e.g.,* category and stability proportions). Thus, even when two releases contain a similar number of packages, their composition can still differ. Using One-Way Anova, we test the null hypothesis that the distributions do not differ across releases. Additionally, we apply a *Tukey* [61] post-hoc test to identify which release pairs exhibit significant differences; as expected, significance is not necessarily observed for every pair of releases.

The Anova test on all categories gives us a $p\text{-value} < 0.05$, which implies that our dataset exhibits significant differences, thus rejecting the *null hypothesis*. Subsequently, we apply the Tukey test to the categories. The results show that the category with the most significant variance compared to the others is data. Additionally, the control and web categories did not overlap in their confidence intervals at any point, indicating a significant difference between them over time.

We also apply the One-Way Anova and Tukey tests to analyze package stability in Stackage. The Anova results in a $p\text{-value} < 0.05$, and the Tukey test shows that a large portion of stability types exhibit a significant variance among them, especially the Experimental stability. In addition, alpha and beta, being such small datasets, did not show significant differences. This leads us to conclude that the Haskell community may not effectively distinguish between what constitutes alpha or beta stability.

## 4. Methodology

Following guidelines of MSR methodologies [10] and similar studies [4,5], we define the methodology for our study. Fig. 4, shows the three stages composing our methodology: (1) retrieving packages, (2) processing package data, and (3) answering the research questions.

The execution of each stage is supported by its own set of tools. The *Scrapy* [62] tool is used to download packages with available

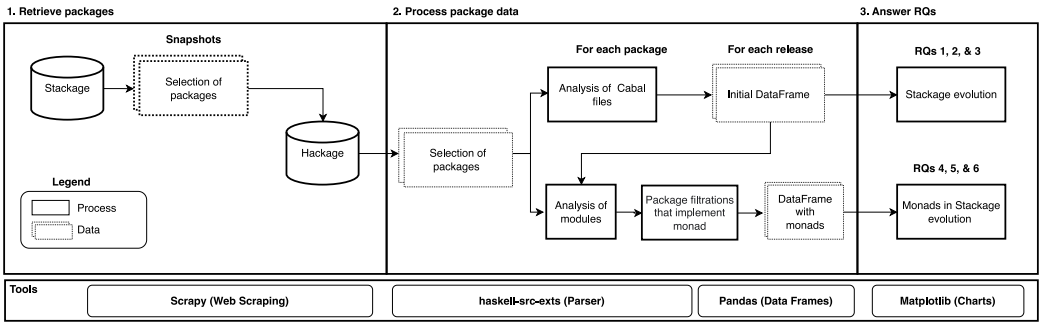



**Fig. 4.** Stackage package retrieval and analysis process.

download links on Stackage. To analyze packages, we use *Pandas* [63], and *Matplotlib* [64]. Finally, we use the *haskell-src-exts* (HSE) package to parse package modules.

We now describe each stage of our methodology.

### 4.1. Retrieving packages

Our study considers all 35 releases published by Stackage between December 2014 and December 2025, comprising 93,377 packages in total. The release periods are fixed and follow a non-uniform schedule (Section 3). Using Scrapy, we automatically download all Haskell packages from all releases. Scrapy provides *Spiders* that allow developers to define how to extract information from a Website. We start by retrieving the package list of a specific Stackage release. As Stackage is a package selection of Hackage, Scrapy directly downloads packages from Hackage. The average time to retrieve all packages in one release, following this process, is one hour.

### 4.2. Processing package data

To process package data, we use DataFrames in Pandas, given its flexibility to (re)structure data. In each package, there is a .cabal file that contains its manifest. We use a Haskell script to statically analyze the manifests and obtain meta-data such as: package dependencies, categories, and the provided modules. To clarify the meta-data processed, we briefly describe the fields that are helpful in retrieving the information to answer the research questions:

**Release:** A unique string in Stackage, which indicates the release version.

**Name:** A unique string in Hackage identifying the package for a release. The same package may exist in multiple releases.

**Version:** A string that follows the format X.Y.Z (Major.Minor.Patch). The same package can exist in different versions across releases.

**Stability:** Unformatted string, which can be added by the developer. This field reports the stability of the package.

**Category:** A list of unformatted strings that describe the categories to which the package belongs. This list must be defined by the developer.

**Dependencies:** List of package names and their version ranges.

**Provided modules:** List of modules publicly exposed by the package.

**Main modules:** List of executable modules in the package. A package can specify multiple executables each with a main module.

With the retrieved meta-data, we analyzed the packages that implement monadic functionality. In particular, we inspect the dependencies of each package to quantify how many and which monad packages are required. To achieve this, we use a Haskell script with HSE to allow us to parse the actions used from each of the selected monad packages: mtl, transformers, monad-control, and free. The selection of these monad packages is the result of the following steps:

**Step 1** Execute a script that revises the title and descriptions of each package that contains the word "monad" and similar variants. This script returns a list of potential monad packages.

**Step 2** Manually filter the list from Step 1., to obtain the packages that actually implement monadic behavior.

**Step 3** Run a script, on the filtered list, that returns the packages that had been imported more than 0.1% from the total number of imports in a release. We implemented this threshold to remove extremely infrequent imports that represent statistical noise rather than meaningful relationships between packages. This ensures that the analysis focuses on representative and consistently used dependencies, while also improving the efficiency of the processing pipeline.

Considering the top monad packages, we add new fields to the DataFrames that describe the monad modules used by each package. With this, we can label and count the use of each monad from the different monad libraries. Each added field corresponds to the use of a specific monad module from the library. We add two general fields for all four packages:

**Imported modules:** A set of package names that are imported into the source code of each package. Each module appears only once, even if it is imported into other files.

**Module-direct flag:** Flag to indicate whether the package depends on one of the four monad packages (*i.e.,* transformers, mtl, monad-control, or free). That is, if one of the four monad packages appears in its dependencies field.

For each package, we add new specific fields that provide information about the monads used from each of the monad modules. Each field is associated with the monad provided by the packages, as shown in Table 3. Whenever a monad is imported into a package we add it up as a use of the monad.

Our empirical study is guided by six Research Questions (RQs), grouped into two complementary dimensions. The first dimension, the first three questions, examines the *structural evolution and maintenance activity* of the repository. The second dimension, the last three questions, explores the *semantic consolidation of core abstractions* through monad packages.

**RQ1** Identify the most imported packages and their stability with the questions: Which packages are the most imported by other packages? Do imported packages have *unstable* or *incompatible* dependencies according to their release?

**Table 3**
Fields added to gather data about the use of required monad packages.

| Field | Monads used |
|---|---|
| mtl package | |
| Cont | Control.Monad.Cont, Control.monad.Cont.Class |
| Error | Control.Monad.Error, Control.Monad.Error.Class |
| Identity | Control.Monad.Identity |
| List | Control.Monad.List |
| RWS | Control.Monad.RWS, Control.Monad.RWS.Class, Control.Monad.RWS.Lazy, Control.Monad.RWS.Strict |
| Reader | Control.Monad.Reader, Control.Monad.Reader.Class |
| Writer | Control.Monad.Writer, Control.Monad.Writer.Class, Control.Monad.Writer.Lazy, Control.Monad.Writer.Strict |
| State | Control.Monad.State, Control.Monad.State.Class, Control.Monad.State.Lazy, Control.Monad.State.Strict |
| Trans | Control.Monad.Trans, Control.Monad.Trans.Class |
| Except | Control.Monad.Except (available only since version 2.2.1) |
| transformers package | |
| Maybe | Control.Trans.Maybe |
| Cont | Control.Trans.Cont |
| Class | Control.Trans.Class |
| RWS | Control.Monad.Trans.RWS, Control.Monad.Trans.RWS.CSP, Control.Monad.Trans.RWS.Lazy, Control.Monad.Trans.RWS.Strict |
| Identity | Control.Trans.Identity |
| Except | Control.Trans.Except |
| Writer | Control.Monad.Trans.Writer, Control.Monad.Trans.Writer.CPS, Control.Monad.Trans.Writer.Lazy, Control.Monad.Trans.Writer.Strict |
| Reader | Control.Monad.Trans.Reader |
| State | Control.Monad.Trans.State, Control.Monad.Trans.State.Lazy, Control.Monad.Trans.State.Strict |
| monad-control package | |
| Monad-control | Control.Monad.Trans.Control |
| free package | |
| Alternative | Control.Alternative.Free, Control.Alternative.Free.Final |
| Applicative | Control.Applicative.Free, Control.Applicative.Free.Fast, Control.Applicative.Free.Final |
| A.Trans | Control.Applicative.Trans.Free |
| Comonad | Control.Comonad.Cofree, Control.Comonad.Cofree.Class, Control.Comonad.Trans.Cofree, Control.Comonad.Coiter |
| Free | Control.Monad.Free, Control.Monad.Free.Ap, Control.Monad.Free.Church, Control.Monad.Free.Class, Control.Monad.Free.TH |
| M.Trans | Control.Monad.Trans.Free, Control.Monad.Trans.Free.Ap, Control.Monad.Trans.Free.Church, Control.Monad.Trans.Iter |

**RQ2** Analyze direct and indirect package dependencies with the question: What is the average number of (in)direct dependencies per package?

**RQ3** Measure package update frequency across releases with the question: How frequently are packages updated?

Regarding the use and evolution of monad packages, we selected the most imported ones in Stackage (*i.e.,* mtl, transformers, monad-control, and free) to address the remaining three research questions:

**RQ4** Describe the evolution of monad packages concerning their presence within Stackage with the question: How have the selected monad packages evolved across releases?

**RQ5** As a complement of **RQ4**, present the use of monad packages from the perspective of the packages in which they are imported with the question: How has the use of the selected monad packages evolved?

**RQ6** Analyze the package evolution that uses monads throughout the releases of Stackage with the question: How many packages that depend on the mtl and transformers packages are added to and removed from Stackage? How many packages that depended on these monad packages stopped their dependencies?

### *4.3. Answers to the research questions*

We now turn our attention to the information sources required to answer the six research questions posit to understand the evolution and use of Stackage and monad packages. To answer these questions, all repository data is structured and processed in DataFrames using Pandas. We briefly describe the process executed on the data frames for each of the questions.

#### *4.3.1.* **RQ1**. *Which packages are imported the most by other Stackage packages? do these packages have unstable or incompatible dependencies according to their Stackage release?*

Using the *dependencies* field, we create a list of dependencies (with their versions) for each package in a release. According to the dependencies, we create a dependency network graph, showing the top 10 packages most used/imported in some releases of the Stackage evolution. Additionally, we extract the (number of) packages that depend on versions of packages that are not available in the specific Stackage release for the package. Finally, using the version information per package, we discovered incompatible dependency versions per release, *i.e.,* at least two packages that depend on the same package with no intersections of ranges in versions.

#### *4.3.2.* **RQ2** *What is the average number of (in)direct dependencies per package?*

We carry out a *Depth First Search (DFS)* [65] in package dependencies for each release used in this study. The DFS algorithm allows us to quantify the average number of (in)direct dependencies per package. Additionally, this analysis also allows us to determine how representative these averages are. The analysis of the package dependencies is relevant to show (1) the repository's dependencies, and (2) the risk of package conflicts as a consequence of some of its dependencies being removed from a specific release, or being discontinued by the authors. Moreover, with the dependency analysis we are able to identify positive effects on package usage, as for example, package reuse and community support to improve maintainability.

#### *4.3.3.* **RQ3**. *How frequently are packages updated?*

Addressing this question involves counting the number of packages that updated their versions for each release. Then, we compare the

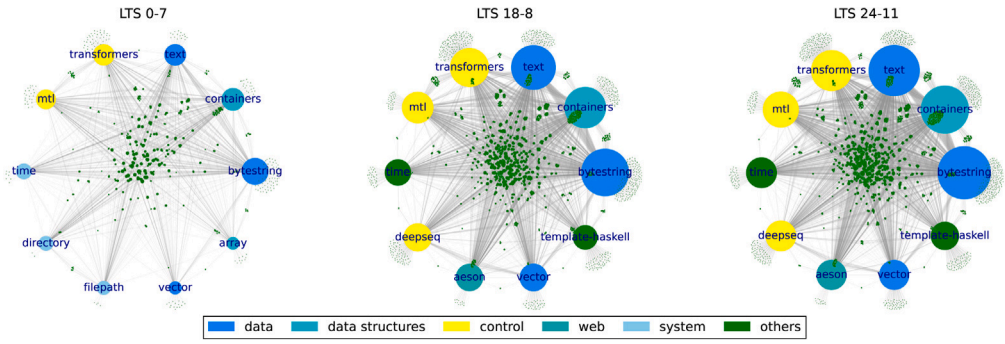



**Fig. 5.** Evolution of package dependencies among the main packages in Stackage **RQ1**.

number of *updated* packages to the number of all packages in a release. Using this result, we can have a proxy for *technical lag* [28] in dependencies, which is helpful to know how frequently bug fixes and new features are included in packages.

*4.3.4.* **RQ4**. *How have the selected monad packages evolved?*

For this question, we parse the source code of each package, across all releases, to count which packages import at least one of the four monad packages: mtl, transformers, monad-control, or free. With this outcome, we can have a sense of the weight of monad behavior in the functionality of Stackage packages.

*4.3.5.* **RQ5**. *How has the use of the selected monad packages evolved?*

To answer this question, we count the modules that are imported from the four selected monad packages through all releases. Then, we compare the number of packages that import these modules to the number of all packages for one selected monad package. With this information, we can get a finer analysis of how monad packages are used, and included as modules for other packages.

*4.3.6.* **RQ6**. *How many packages that depend on the* mtl *and* transformers *packages are added to and removed from Stackage? How many packages that depended on these monad packages stopped their dependencies?*

To answer the first part of this question, we count each added or removed package that imported the mtl or transformers modules in a release. For the second part of the question, we follow a similar process, in this case counting packages that used the mtl or transformers modules and then stopped using them. To answer this question, we did not analyze the monad-control and free monad modules because their participation in each release is not significant.

## 5. Results to the research questions

We now turn our attention to the six research questions posit in this study, intending to understand the evolution and use of Stackage and its monad packages. To answer these questions, we process the information gathered from packages as Pandas DataFrames (Section 4.2), as described in Section 4.3. We maintain the same scale for charts related to a specific RQ to facilitate the comparison of results.

*5.1.* **RQ1**. *Which packages are imported the most by other Stackage packages? Do these packages have unstable or incompatible dependencies according to their Stackage release?*

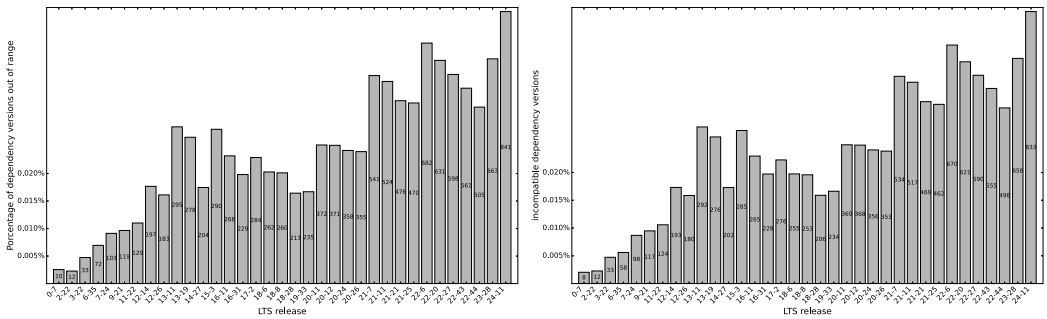
Identifying the *most imported* packages in Stackage is not merely a descriptive task; it provides insight into the *critical dependencies* that constitute the structural backbone of the ecosystem. Prior studies across ecosystems such as npm, CRAN, and PyPI have shown that the *topology of dependency networks* significantly influences ecosystem resilience, the propagation of *technical lag*, and vulnerability exposure [2,3,37]. Mapping these "infrastructure nodes" within Stackage allows maintainers to focus curation and testing efforts where instability could have a disproportionate cascading effect.

Using the *dependencies* field in the data frames, we created a list of dependencies (and their versions) for each package in every release. Fig. 5 presents the dependency network for three representative releases of Stackage: the first LTS release in our dataset, a mid-point in the timeline, and the most recent LTS considered. At first glance, the number of dependencies increases along with the growth of the repository. Furthermore, the top seven packages (*i.e.,* time, mtl, transformers, text, containers, bytestring, and vector) maintain their dependency ranks throughout the evolution of Stackage. Although mtl and transformers are not the absolute top by count, they remain widely used across releases, confirming their role as essential abstractions for a large portion of the ecosystem. Overall, these packages belong to different categories, showing that Stackage's evolution maintains a diverse but stable dependency core. The category labels shown in Fig. 5 are drawn from each package's Hackage metadata at release time; since categories are provided by maintainers and can be updated, the same package may appear under different categories across releases (e.g., time was tagged as "system" in 2014 but as "others" in later releases), and some categories present in the network visualization (such as "data structures" and "system") may not appear prominently in Fig. 3 if only a few packages are categorized that way.

When reasoning about dependency versions, two complementary observations emerge. First, Fig. 6(a) shows the number and percentage of packages whose dependencies refer to versions not available in their respective LTS—that is, *unstable dependencies*. Across all releases, the fraction of such unstable dependency versions remains extremely small, representing only a tiny percentage of all declared dependency versions, and fluctuates mildly over time rather than exhibiting a strong monotonic trend. Second, Fig. 6(b) displays packages that depend on the same package but specify disjoint version ranges, leading to *incompatible dependencies*. For this analysis, we initially used the first revision of each .cabal file.

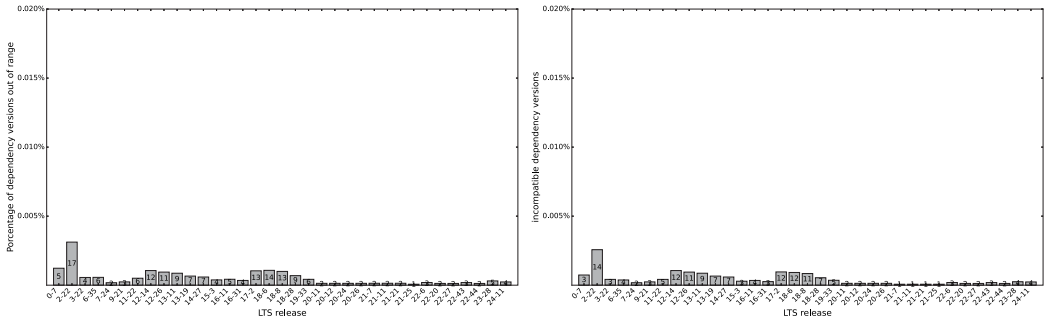
We shared the preliminary results [8] of **RQ1** with the Stackage community in a public GitHub discussion, "Safety issues in Stackage?" [66]. Community maintainers pointed out that each package's .cabal file can have multiple *revisions* in addition to version updates, a factor that had not been originally considered. We therefore extended our analysis to include the latest revision of each manifest. With this refinement, actual unstable and incompatible dependencies become more visible, with a tendency to reduce the amount of such packages recent LTS releases (although some still remain), as depicted in Fig. 7, although they still account for only a very small proportion of all dependency versions.

*Qualitative inspection.* To better understand the causes of instability, we manually inspected three of the top imported packages (mtl, containers, and time). We found that none of the issues were related to semantic API breakages but rather to *metadata inconsistencies*: missing or outdated version constraints, or misalignment between Hackage versions and




(a) Unstable dependencies per release (b) Incompatible dependencies per release

**Fig. 6.** Unstable and incompatible dependencies per Stackage release using the first .cabal files **RQ1**.




(a) Revised unstable dependencies per release (b) Revised incompatible dependencies per release

**Fig. 7.** Unstable and incompatible dependencies per Stackage release using the revised version of .cabal files **RQ1**.

their inclusion in a particular LTS release. Importantly, even for these popular packages, we did not observe evidence that instability stems from a lack of semantic maturity; instead, the observed unstable or incompatible cases arise from the way version constraints and snapshot availability interact at the ecosystem level. These findings are consistent with results reported in other ecosystems, where dependency fragility often arises from insufficient constraint synchronization and manual release coordination [1,67].

Furthermore, we observe that while it is the case that packages with incompatible dependencies also present unstable dependencies, as it is the case in releases 20-11 and 20-15, this not need to be true for all packages. Often is the case that a package presents incompatible dependencies and no unstable dependencies, and viceversa. For example, in release 17-2 the bugzilla-redhat package contains unstable dependencies, but no incompatible dependencies. Moreover, the packages that present such instability often endure through multiple releases with the same problems, as it is the case of the bench-show and brik packages between releases 20-11 and 2015. However this is not generalizable to all releases. For example, there is a decrease of instability issues between releases 2-22 and 3-22, which may be due to the packages deleted between the releases, 361 in total (with 75 new packages added in release 3-22). In contrast, the increase in instability issues between releases 16-31 and 17-2 may be due to the increase the amount of packages introduced for release 17-2 (259 new packages, while 123 packages were deleted), as it is the case for example of the bugzilla-redhat package. Not, however, that not all introduced packages introduce such safety issues.

Such metadata-driven instability reinforces the need for *automated constraint checking tools* and continuous validation pipelines to ensure reproducibility in curated repositories.

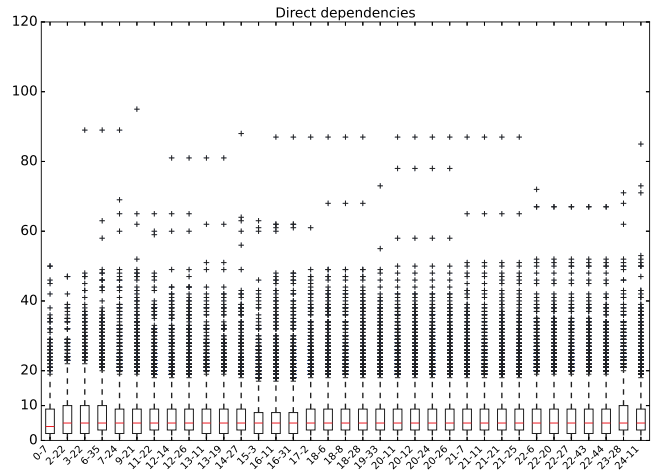
*Interpretation.* From an ecosystem perspective, the most imported packages in Stackage act as *high-impact nodes* in the dependency network: their stability and maintenance practices strongly affect repository health. Monitoring these critical dependencies can help prioritize quality assurance and dependency audits, reducing the risk of transitive failures and long-term drift. In line with studies on other software ecosystems [68–70], we interpret these "core packages" as both an asset and a vulnerability point within Stackage's dependency structure. In the discussion section (Section 6), we return to these core packages as part of a broader reflection on structural fragility and curated stability in the repository.

### 5.2. RQ2. *What is the average number of direct and indirect dependencies per package?*

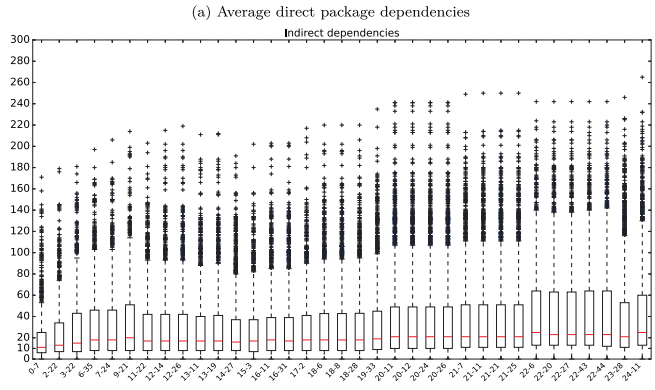
Understanding the number of direct and indirect dependencies in a repository is essential to characterize its *structural complexity* and potential *fragility*. Prior research has shown that dependency chains serve as strong predictors of *technical lag*, maintainability, and transitive risk propagation in software ecosystems [1–3]. A higher number of dependencies increases the maintenance surface and the effort required to keep packages synchronized, while a lower number may indicate reduced reuse and weaker integration within the community. This balance directly relates to the long-term maintainability principles observed in software evolution laws [71,72].

To answer this question, we use box plots to illustrate the distribution of both direct and indirect dependencies for each Stackage release. Fig. 8(a) shows that the typical number of *direct dependencies* per package remains relatively stable over time, with median values around five to seven across releases. Most packages depend on fewer than 15 direct packages, but there is a clear long tail: some packages accumulate between 20 and 50 direct dependencies, and a small number of outliers approach roughly 100. This skewed distribution indicates that while the mean remains stable, a small subset of highly connected packages could play a disproportionate role in dependency propagation.

Fig. 8(b) confirms that the number of *indirect dependencies* per package is substantially higher than that of direct dependencies. Median




(a) Average direct package dependencies




(b) Average indirect package dependencies

**Fig. 8.** Number of direct and indirect package dependencies per release **RQ2**.

values lie roughly between 10 and 20 indirect dependencies per package, and the interquartile range widens over successive releases. Many packages accumulate more than 70 indirect dependencies, and some exceed 200, underscoring that simple averages are not fully representative of the highly skewed distribution. While outliers represent only a small fraction of packages, they are precisely the cases that can concentrate transitive exposure: in a dependency graph, an issue in a highly central package can propagate to many downstream packages through long chains, as reported in prior ecosystem studies on dependency fragility and technical lag [1–3].

*Interpretation.* The overall averages of direct and indirect dependencies do not vary significantly across releases, suggesting that Stackage achieves a *curated balance between reuse and stability*. Related work reports that uncontrolled growth of dependency graphs in large, uncurated ecosystems can lead to fragility and versioning conflicts [1,2]. In Stackage, most packages remain within a stable dependency range, indicating that the curation process effectively limits dependency sprawl. Nevertheless, the presence of long-tailed distributions highlights a small fraction of packages that concentrate dependency risk and should be monitored closely. We interpret these patterns within the scope of Stackage; a direct empirical comparison against Hackage is left as future work.

*Practical implications.* We propose using dependency metrics as *operational indicators* to support repository governance: (1) flagging packages with extremely long dependency chains as potential risk hotspots; (2) prioritizing those packages during dependency validation and regression testing; and (3) employing automated update mechanisms, such as bot-driven pull requests, to mitigate lag in packages that exhibit high transitive reach [67]. Overall, these findings reinforce the need for continuous dependency management and monitoring, particularly for highly connected packages that can act as amplifiers of instability within the ecosystem. In the discussion, we build on these observations to characterize how such dependency structures shape the long-term evolution and maintainability of curated repositories like Stackage.

### 5.3. RQ3. *How frequently are packages updated?*

The frequency with which packages are updated reflects both the *maintenance activity* of developers and the ecosystem's ability to evolve while avoiding the accumulation of *technical lag*. A lower update frequency may indicate stability and maturity, whereas a sharp decline can also suggest reduced developer attention or maintenance fatigue [2, 71]. Understanding this dynamic is essential for evaluating the long-term maintainability of a curated repository such as Stackage.

To answer this question, we count the number of packages that changed version between consecutive releases and compared it to the total number of packages in each release. Fig. 9 shows the resulting percentage of updated packages per release. This percentage serves as a proxy for *dependency freshness* and can be interpreted as an indicator of *technical lag* [28]. Early in the observed period, a higher proportion of packages are updated between consecutive releases, whereas in the later part of the timeline the proportion is generally lower, with only occasional spikes, reflecting an overall slower pace of change.

*Interpreting the decline.* The decrease in update frequency aligns with two temporal trends. First, the *interval between releases* shortened over time; from an average of six months to roughly three leaving maintainers less time to prepare and submit updated versions. Second, several core packages appear to have reached a higher level of *API stability*, leading to fewer required updates. This pattern is consistent with the later stages of software evolution described by Lehman's laws, where systems tend toward equilibrium and slower structural change [71,72]. However, a drop in update frequency can also signal the emergence of dormant or abandoned projects within the ecosystem, which may gradually accumulate technical lag [1,3].

*Temporal normalization.* To clarify whether the decline is structural or an artifact of shorter release cycles, we recomputed the metric using fixed six-month windows rather than release boundaries. The adjusted analysis confirms a *genuine reduction in update activity* over time, suggesting that even accounting for interval differences, the ecosystem has slowed its pace of evolution. This slowdown can reflect either increased maturity or a shift in community engagement; both interpretations have implications for repository health and reproducibility [73,74].

*Implications for maintenance and governance.* A declining update rate emphasizes the need for mechanisms that continuously assess package freshness and identify outdated dependencies. Curated repositories could mitigate stagnation by introducing *lag dashboards*, periodic dependency audits, or automated notifications encouraging maintainers to revalidate and republish packages. Combining these measures with targeted reviews of critical dependencies would help ensure that stability in Stackage does not come at the cost of long-term maintainability. Overall, while the observed slowdown indicates consolidation and maturity, it also highlights the importance of proactive maintenance policies to prevent silent erosion of dependency quality across the ecosystem.

### 5.4. RQ4. *How have the selected monad packages evolved?*

Monad libraries such as `mtl`, `transformers`, `monad-control`, and `free` represent fundamental abstractions in the Haskell ecosystem, providing reusable interfaces for effect management and modular program design. Understanding their evolution across Stackage releases offers insight into the *semantic stability* and *abstraction maturity* of the ecosystem.

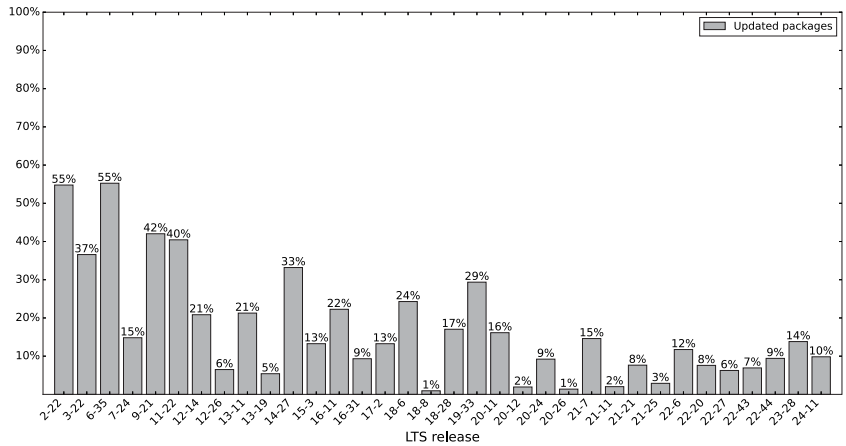



**Fig. 9.** RQ3. Percentage of updated packages per release.

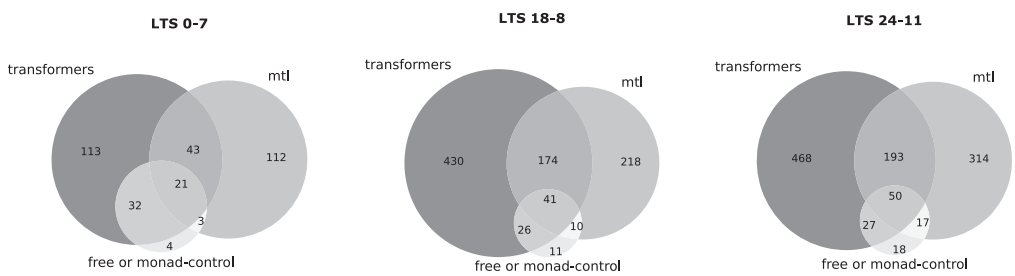



**Fig. 10.** Usage of monad packages in Stackage RQ4.

Tracking how these core libraries are imported over time can reveal whether developers converge on standardized patterns or continue to explore alternative abstractions.

To answer this question, we parsed the source code of each package to determine whether it imported any of the four selected monad libraries. Fig. 10 presents the distribution of these imports at different points in Stackage's evolution. Overall, we observe a gradual growth in the use of monad packages and their combinations across the repository, with the majority of packages depending on either mtl or transformers, or both. Concretely, the number of packages importing at least one of the selected monad libraries increases from 328 in the first LTS snapshot shown (LTS 0–7) to 910 in the mid-point (LTS 18–8) and 1087 in the most recent snapshot (LTS 24–11). The overlap between mtl and transformers also grows over time (43 + 21 packages in LTS 0–7 vs. 174 + 41 in LTS 18–8 and 193 + 50 in LTS 24–11), indicating that co-usage becomes more common as the ecosystem matures. In contrast, monad-control and free are adopted in fewer projects, reflecting their more specialized use cases.

*Interpretation.* The persistent dominance of mtl and transformers over more than a decade of releases suggests that Stackage developers rely on a *stable and well-understood abstraction layer*. This stability aligns with ecosystem maturity: core functional components evolve more slowly once community conventions solidify. Meanwhile, the limited adoption of alternative libraries such as free indicates that experimentation with higher-order monad abstractions remains marginal. Similar evolutionary patterns – where ecosystems consolidate around a few foundational abstractions – have been observed in other programming-language communities [1,69].

*Implications.* The results highlight that Stackage's semantic core is not only stable but also cohesive, suggesting that its curation effectively promotes convergence toward common programming patterns. For maintainers, these findings emphasize the importance of sustaining backward compatibility in the dominant libraries (mtl and transformers), as their consistency underpins the long-term reproducibility and interoperability of the broader ecosystem.

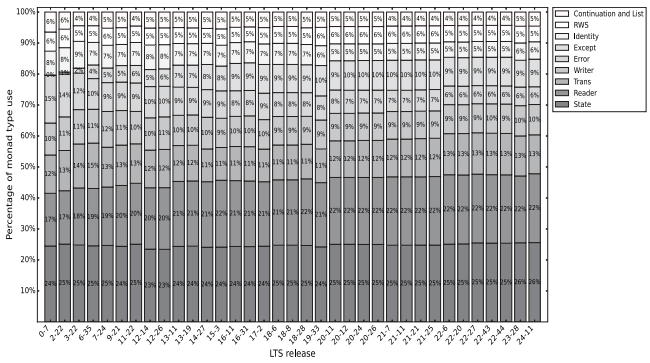
### 5.5. RQ5. *How has the use of the selected monad packages evolved?*

To answer this question, we counted the number of imported modules from the selected monad packages (mtl, transformers, monad-control, and free) in each release. Tracking module-level imports allows us to explore the *semantic evolution* of these abstractions, showing which functional constructs developers favor and how they shift over time. The stability or substitution of specific monads can reveal how the community consolidates patterns of state and error handling within the ecosystem.

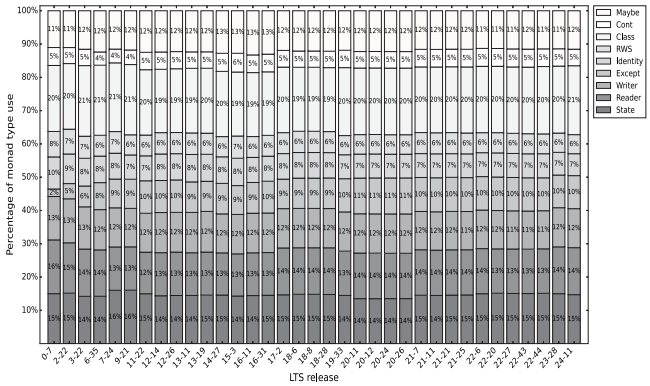
Figs. 11(a) and 11(b) display the evolution of the most imported modules in mtl and transformers, respectively. For both libraries, the State and Reader monads maintain a strong and consistent presence compared to other abstractions. This trend likely reflects developers' preference for expressing *stateful behavior* and environment passing, two central idioms in functional programming. The persistent use of these modules over more than a decade of releases suggests that their design has achieved a stable balance between expressiveness and simplicity. Across releases, the distribution of the most imported modules remains fairly stable overall: the dominant modules typically stay within a narrow band of percentages, and changes tend to be gradual rather than abrupt.

Within mtl, Fig. 11(a)] shows an inverse relationship between the use of Error and Except. This pattern aligns with the gradual *substitution of older error-handling abstractions* by more modern and type-safe alternatives, as Except can serve as a direct replacement for Error in most scenarios. In contrast, the distribution of modules in transformers remains relatively stable after the early releases, with only modest fluctuations through the most recent LTS releases considered, indicating that once mature interfaces are established, developers rarely diverge from them.

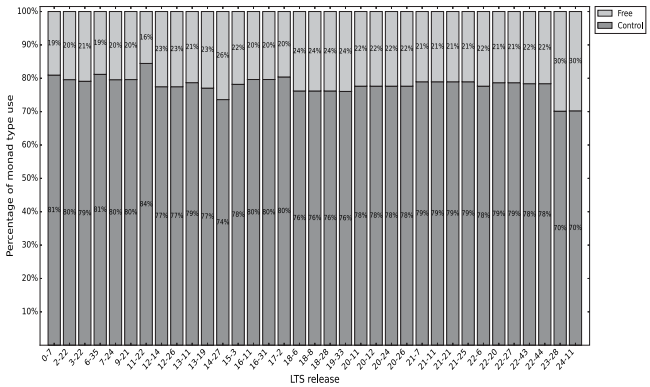
Given the limited participation of monad-control and free, Fig. 11(c) summarizes their adoption across releases. We observe that monad-control




(a) Distribution of monads used in mtl over time.




(b) Distribution of monads used in transformers over time.




(c) Distribution of monad-control and free package usage over time.

**Fig. 11.** Monad packages used in Stackage per release RQ5.

appears more frequently than free, which may be explained by its closer integration with conventional monadic transformers and its practical role in managing lifted computations. The figure also shows that monad-control consistently represents the majority of these imports (roughly three quarters across releases), while free accounts for the remaining quarter, reinforcing that free remains a niche choice in Stackage.

*Interpretation.* The consistent prevalence of mtl and transformers indicates that Stackage developers converge on a *shared semantic foundation* for effect composition. This consolidation mirrors the maturity observed in other programming ecosystems, where a few dominant abstractions emerge and become de facto standards [1,69]. The stable adoption of these modules supports the idea that Stackage's curation not only preserves dependency stability but also encourages conceptual uniformity across projects.

*Implications.* From a maintenance perspective, the slow and steady evolution of monad usage demonstrates that *semantic stability* accompanies syntactic and dependency stability in the ecosystem. For maintainers, ensuring backward compatibility and documentation quality in these core libraries is crucial, as even minor interface changes could affect a substantial portion of the ecosystem that depends on these abstractions.

### 5.6. RQ6. *How many packages that depend on the mtl and transformers packages are added to and removed from Stackage? How many packages that depended on these monad packages stopped their dependencies?*

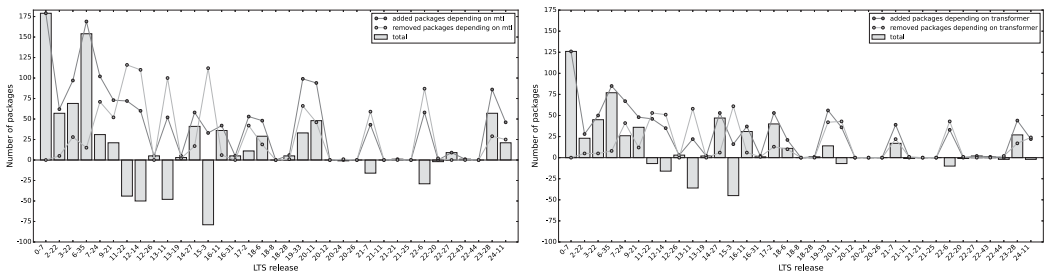
Analyzing how dependencies to mtl and transformers appear and disappear over time provides insight into the *stability and persistence* of core abstractions within the ecosystem. While most projects consistently rely on these foundational libraries, the rate at which packages adopt or drop them reflects how developers adjust their designs and whether the ecosystem converges toward long-term stability.

Fig. 12 shows the number of added and removed packages that import modules from mtl (Fig. 12(a)) and transformers (Fig. 12(b)) across all releases. Packages depending on monad-control and free were excluded due to their limited participation in Stackage. Both figures show that the early releases exhibit a high influx of packages adopting these dependencies, followed by periods of fluctuation and gradual consolidation. Across consecutive LTS releases, the net change (*total*) alternates between positive and negative values, indicating that adoption and removal coexist throughout the timeline. However, the magnitude of these swings is generally moderate, and later releases tend to exhibit smaller net changes, consistent with consolidation in a mature curated ecosystem.

Fig. 13 presents the number of packages that started or stopped using mtl (Fig. 13(a)) and transformers (Fig. 13(b)) between consecutive releases. In both cases, only a small fraction of packages change their dependency status between consecutive releases: most transitions involve fewer than 10 packages, with occasional peaks around a dozen, which remains well under 1% of a typical release. These minor fluctuations suggest that once a package adopts either mtl or transformers, it rarely removes them in later releases. Such persistence indicates that these libraries occupy a stable and well-integrated position within the Stackage ecosystem.

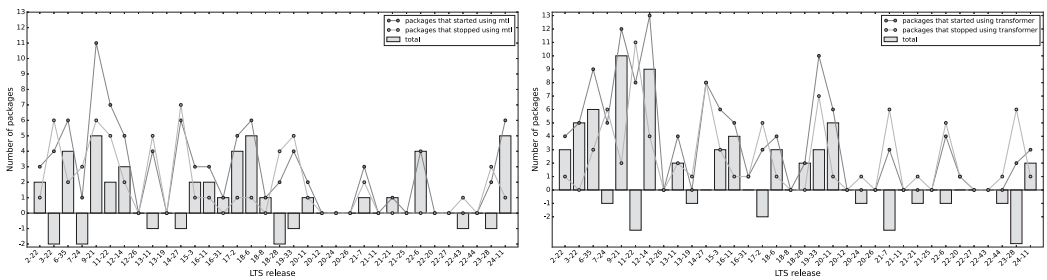
*Interpretation.* Both mtl and transformers show similar long-term behavior: they experience early expansion phases, followed by stabilization once they become entrenched in the dependency graph. This pattern suggests that, for these two foundational abstractions, the ecosystem undergoes an initial period of adoption and alignment, after which their role becomes firmly consolidated. We do not claim that the same trajectory necessarily applies to other libraries; extending this comparative analysis to additional (non-monad) packages is an interesting direction for future work. The coexistence of both packages also reflects their *complementary functionality*: developers often employ them together to ensure *compatibility* with other libraries and tooling already built on top of these abstractions (for example, testing frameworks such as QuickCheck).

*Implications.* The observed persistence and co-dependence of mtl and transformers imply that these libraries form a *semantic backbone* of Stackage. Their continued presence across releases reinforces the idea that the ecosystem prioritizes conceptual and behavioral consistency over radical change. For maintainers, this highlights the need to safeguard the interoperability between both packages and to ensure that future revisions remain backward compatible, as any disruptive modification could propagate through a substantial portion of the dependency network. Overall, the data indicate that Stackage's evolution balances stability with limited, low-impact churn—an expected characteristic of a mature and curated ecosystem [1,69].




(a) mtl dependent packages. (b) transformers dependent packages.

**Fig. 12.** Number of added and removed packages dependent on mtl or transformers.




(a) Stopped dependencies to mtl. (b) Stopped dependencies to transformers.

**Fig. 13.** Number of packages that stopped importing mtl or transformers dependencies.

## 6. Discussion

The results of our study portray the evolution of Stackage from both the structural and semantic perspectives. Beyond describing quantitative trends, this section interprets what these patterns reveal about the dynamics of curated software ecosystems, linking them to theories of software evolution and ecosystem maintainability. We organize the discussion around three dimensions: *structural evolution*, *semantic consolidation*, and *curated stability*.

### 6.1. Structural evolution: controlled growth and fragility

The analysis of dependency networks, complexity, and update frequency (RQ1, RQ2, and RQ3) reveals that Stackage evolves through *controlled growth*. Prior work has documented that, in large uncurated ecosystems, dependency networks may expand rapidly and introduce fragility and conflicts [1,2]. In our study, we focus on Stackage as a curated ecosystem and observe how its governance moderates these dynamics. The stable distribution of direct and indirect dependencies across releases (RQ2) indicates that most packages remain within a moderate dependency range, even as the repository grows. This behavior illustrates a trade-off between reuse and fragility: curation minimizes uncontrolled proliferation but cannot eliminate the systemic risk caused by a few highly connected packages. The long-term reduction in update frequency suggests increasing maturity and stability, yet it also hints at potential maintenance fatigue and technical lag—phenomena described in Lehman's laws of software evolution [71]. Together, these findings show that curated ecosystems can keep dependency structures under control while still accumulating risk in a small subset of core packages, which become focal points for governance and quality assurance. Overall, Stackage demonstrates that deliberate governance can slow the accumulation of complexity without halting change.

> *Curated ecosystems evolve through deliberate constraint: they grow slowly, avoid chaotic dependency expansion, and trade speed for long-term maintainability.*

### 6.2. Semantic consolidation: convergence toward stable abstractions

The examination of monad libraries and their usage patterns (**RQ4** & **RQ6**) highlights the *semantic stability* of Stackage. Once developers converge on a shared set of abstractions (mtl and transformers), subsequent evolution favors refinement over replacement. This convergence echoes the "abstraction maturity" observed in other ecosystems [69], where communities standardize around a few idiomatic solutions that become de facto norms. The gradual substitution of outdated constructs (e.g., Error replaced by Except) shows that innovation occurs incrementally and conservatively, guided by backward compatibility and collective consensus. The low churn in dependencies on these libraries confirms that semantic cohesion and structural stability reinforce each other.

> *Mature ecosystems consolidate around stable abstractions: evolution continues, but within well-defined semantic boundaries that preserve interoperability and learning.*

### 6.3. Curated stability and maintainability

Bringing both perspectives together, the Stackage case illustrates that *stability does not mean stagnation*. Curated ecosystems embody a specific evolutionary mode—neither the rapid experimentation of open repositories nor the rigidity of frozen systems. Instead, they practice *disciplined evolution*: steady refinement governed by explicit quality gates and social mechanisms of trust. The dependency metrics analyzed in **RQ1** and **RQ2** provide concrete signals for this governance: they highlight high-impact packages, long dependency chains, and potential risk hotspots that curators can monitor and prioritize. In practical terms, such metrics can support policies such as targeted regression testing for core packages, periodic audits of unstable or incompatible dependencies, and automated mechanisms to surface technical lag before it accumulates. This model supports predictability and reproducibility, aligning with the needs of developers who value reliability over novelty. However, it also requires continuous stewardship to prevent quiet obsolescence and to sustain community engagement [3]. In this

sense, curation can be understood as a social process of co-evolution between maintainers and their tools. We note that our discussion interprets patterns within Stackage; a direct quantitative comparison against Hackage is beyond the scope of this paper and remains an avenue for future work.

> *Curation transforms software evolution into a collective act of governance: stability emerges from continuous, coordinated, and transparent refinement rather than from the absence of change.*

### 6.4. Overall reflection

By combining quantitative and qualitative perspectives, this study contributes an empirical view of how a curated ecosystem evolves over more than a decade (December 2014–December 2025). Stackage demonstrates that maintainability in software ecosystems stems from moderation: it thrives by controlling growth, reusing stable abstractions, and promoting long-term reproducibility. These insights extend existing theories of software evolution [71] to modern, community-driven repositories, where the laws of change are not only technical but also social.

## 7. Related work

We follow guidelines from the MSR [10] community to conduct our exploratory study of Stackage. This section reviews prior work on (1) the evolution of software ecosystems and package repositories, (2) dependency management and fragility across ecosystems, and (3) studies focusing on the Haskell community.

*Ecosystem evolution and repository analysis.* A substantial body of work has examined the evolution of software ecosystems and package repositories across different programming languages. For instance, Plakidas et al. [21] provide an exploratory characterization of the CRAN repository, highlighting how package distribution shapes the identity of the R developer community. Kikas et al. [19] investigate dependency networks in JavaScript, Ruby, and Rust repositories, analyzing how updating or removing critical packages can propagate instability. Similarly, Chinthanet et al. [39] study lag in npm [11] dependencies, focusing on the time between vulnerability discovery and patch adoption. These works inspired our quantitative approach to characterizing the dependency network structure in Stackage, and our research questions **RQ2** and **RQ3** build upon their analytical frameworks.

Beyond these descriptive studies, recent research has deepened the understanding of long-term ecosystem dynamics. Decan et al. [1] build a large-scale comparison of dependency network evolution in seven ecosystems, identifying systematic differences in growth patterns, reuse, and transitive dependency chains. Zerouali et al. [2] examine the impact of outdated and vulnerable dependencies on ecosystem health, highlighting the prevalence of maintenance debt. Prana et al. [3] expand outdated vulnerable dependencies by empirically analyzing dependency fragility through dependency-chain length and transitive risk, providing concrete metrics for ecosystem resilience. These studies shape the interpretation of our Stackage results, where we discuss fragility and stability trends in a curated ecosystem.

*Dependency management and coordination.* Also related to our presentation is the research efforts focusing on the way developers and maintainers manage dependencies and coordinate updates. Mirhosseini and Parnin [67] examine how dependency updates affect the stability of software systems, revealing that many projects delay updates due to fear of breaking changes. Serebrenik and Mens [69] emphasize the socio-technical nature of ecosystem evolution, arguing that coordination, governance, and curation mechanisms play a key role in mitigating fragility. Recent work by Bommarito and Bommarito [38] and Jia et al. [42] extend this perspective to curated and domain-specific ecosystems, showing that curation policies influence dependency health and synchronization. In our study, we adopt these insights to contextualize Stackage as a curated repository designed to balance automation with manual validation.

*Studies on the haskell ecosystem.* While numerous studies exist for npm, CRAN, and other repositories, few have explored Haskell's package ecosystems in depth. Figueroa [4], Figueroa et al. [5] investigate the usage and evolution of monads in Hackage, while Bezirgiannis et al. [75] analyze the way developers use generic language constructs and tools in Haskell. Morris [6] studied practical programming patterns extracted from Hackage. However, none of these works examined the longitudinal evolution of the ecosystem, nor the dependency stability across multiple curated releases. To the best of our knowledge, our work provides the first longitudinal and mixed-method empirical study of the Stackage repository, linking its curated release model to ecosystem health, dependency stability, and maintenance coordination.

## 8. Conclusion and future work

Programming language communities are usually built around repositories that allow developers to access and share packages. Normally, the relevance of repositories in the community is associated with their *stability*. Moreover, through the analysis and evaluation of the evolution of package repositories, we demonstrated that it is possible to identify the identity forged in the community. An example is our analysis of the Stackage packages, a curated repository for stable Haskell packages in the Hackage repository. In Stackage, maintainers make available LTS releases that contain a set of packages with specific versions. Although Stackage is widely used (in industry, as its main target), there are currently no empirical studies on how this repository has evolved. Specifically, we focus on using a defining characteristic in Haskell: monads. This paper is the first large-scale analysis of the evolution of the Stackage repository: 35 LTS releases (December 2014–December 2025), comprising 93,377 packages. The main findings of our work highlight that: First, we quantify the presence of release-time dependency-version inconsistencies (unavailable versions and incompatible constraints) and show that `.cabal` revisions mitigate most cases, underscoring the importance of revision mechanisms and constraint hygiene in curated releases. Second, the `mtl` and `transformers` packages are among the top 10 most used packages in Stackage. Third, as in previous studies [4,5], we conclude that repository maintainers should be careful when providing unstructured fields for metadata such as *stability* or *category* because developers do not follow a protocol to provide this kind of information. Finally, for the `mtl` and `transformers` monad packages, the `State` and `Reader` monads are the most used in Stackage.

Beyond these ecosystem "hotspots", we also observe that Stackage maintains a relatively stable distribution of direct and indirect dependencies across releases, while the fraction of packages updated between consecutive LTS releases generally decreases over time (with occasional fluctuations), consistent with consolidation in a mature curated ecosystem.

The findings presented in this paper are firmly based on the analysis of dependencies and modules imported in package releases. This kind of analysis obfuscates a fine-grained use of functions provided by modules in packages, including monads. This is because this analysis does not allow us to know what and how imported functions are really being used (if they are used at all). We plan to complete a static source code analysis to answer the previous questions. As a result, this analysis may provide insights into why specific packages (*e.g.,* `mtl`) are being imported.

## CRediT authorship contribution statement

**Paul Leger:** Funding acquisition, Formal analysis, Data curation, Conceptualization. **Felipe Ruiz:** Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **David Moreno-Lumbreras:** Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Nicolás Sepúlveda:** Methodology, Investigation, Funding acquisition, Formal analysis, Data

curation, Conceptualization. **Ismael Figueroa:** Funding acquisition, Formal analysis, Data curation, Conceptualization. **Nicolás Cardozo:** Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

I have shared a link to access to data.

## References


[1] A. Decan, T. Mens, E. Constantinou, On the evolution of technical lag in the NPM package dependency network, in: IEEE International Conference on Software Maintenance and Evolution, ICSME, Madrid, Spain, 2018, pp. 404–414.

[2] A. Zerouali, T. Mens, J.M. Gonzalez-Barahona, G. Robles, A formal framework for measuring technical lag in component repositories—and its application to NPM, J. Softw.: Evol. Process. 31 (2018) e2157.

[3] G.A.A. Prana, A. Sharma, D. Lo, Out of sight, out of mind? How vulnerable dependencies affect open-source projects, Empir. Softw. Eng. 26 (2023) 1–34.

[4] I. Figueroa, A preliminary assessment of how monads are used in haskell, in: Proceedings of the Brazilian Symposium on Programming Languages, Fortaleza, Brazil, 2017, pp. 1–7.

[5] I. Figueroa, P. Leger, H. Fukuda, Which monads haskell developers use: An exploratory study, Sci. Comput. Program. 201 (2021) 102523.

[6] J.G. Morris, Experience report: Using hackage to inform language design, in: Proceedings of the Third ACM Haskell Symposium on Haskell, Baltimore, Maryland, USA, 2010, pp. 61–66.

[7] haskell.org, Haskell cabal, 2024, http://www.haskell.org/cabal. Last visited: 01/04/2024.

[8] P. Leger, F. Ruiz, N. Sepúlveda, I. Figueroa, Evolution of a Haskell repository and its use of monads: An exploratory study of stackage, in: Proceedings of the Symposium on Applied Computing, Virtual Conference, 2022, pp. 1475–1482.

[9] F. Ruiz, N. Sepulveda, I. Sepulveda, P. Leger, Script to process the stackage repository, 2025, https://github.com/pragmaticslaboratory/stackage-evolution. Last visited: 12/19/2025.

[10] H. Hemmati, S. Nadi, O. Baysal, O. Kononenko, W. Wang, R. Holmes, M.W. Godfrey, The MSR cookbook: Mining a decade of research, in: Proceedings of the Working Conference on Mining Software Repositories, San Francisco, USA, 2013, pp. 343–352.

[11] npm, Build amazing things, 2024, https://www.npmjs.com. Last visited: 01/04/2024.

[12] PyPI, Find, install and publish python packages with the python package index, 2024, https://pypi.org. Last visited: 01/04/2024.

[13] NuGet, Nuget is the package manager for .NET, 2024, https://www.nuget.org. Last visited: 01/04/2024.

[14] Hackage, A central package repository for haskell, 2024, http://hackage.haskell.org. Last visited: 01/04/2024.

[15] Stackage, A curated repository for stable haskell packages, 2024, https://www.stackage.org. Last visited: 01/04/2024.

[16] A. Decan, T. Mens, M. Claes, P. Grosjean, On the development and distribution of R packages, in: Proceedings of the 2015 European Conference on Software Architecture Workshops, Prague, Czech Republic, 2015, pp. 1–6.

[17] A. Decan, T. Mens, E. Constantinou, On the Evolution of Technical Lag in the npm Package Dependency Network, IEEE, 2018, pp. 404–414.

[18] Y. Hu, J. Zhang, X. Bai, S. Yu, Z. Yang, Influence analysis of github repositories, SpringerPlus 5 (2016) 1268.

[19] R. Kikas, G. Gousios, M. Dumas, D. Pfahl, Structure and evolution of package dependency networks, in: Proceedings of the International Conference on Mining Software Repositories, Buenos Aires, Argentina, 2017, pp. 102–112.

[20] N. Munaiah, S. Kroh, C. Cabrey, M. Nagappan, Curating GitHub for engineered software projects, Empir. Softw. Eng. 22 (2017) 3219–3253.

[21] K. Plakidas, D. Schall, U. Zdun, Evolution of the R software ecosystem: Metrics, relationships, and their impact on qualities, J. Syst. Softw. 132 (2017) 119–146.

[22] S. Jansen, Measuring the health of open source software ecosystems: Beyond the scope of project health, Inf. Softw. Technol. 56 (2014) 1508–1519.

[23] O. Franco-Bedoya, D. Ameller, D. Costal, X. Franch, QuESo - A quality model for open source software ecosystems, in: Proceedings of the 9th International Conference on Software Engineering and Applications, Vienna, Austria, 2014, pp. 209–221.

[24] M. D'ambros, H. Gall, M. Lanza, M. Pinzger, Analysing Software Repositories to Understand Software Evolution, Springer Berlin Heidelberg, 2008, pp. 37–67.

[25] S. Romano, M. Caulo, M. Buompastore, L. Guerra, A. Mounsif, M. Telesca, M. Baldassarre, G. Scanniello, G-repo: a tool to support MSR studies on GitHub, in: IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, Honolulu, USA, 2021.

[26] R. Dyer, H. Nguyen, H. Rajan, T. Nguyen, Boa: A language and infrastructure for analyzing ultra-arge-scale software repositories, in: 35th International Conference on Software Engineering, ICSE, San Francisco, USA, 2013.

[27] W. Shang, B. Adams, A. Hassan, Using pig as a data preparation language for large-scale mining software repositories studies: An experience report, J. Syst. Softw. 85 (2012) 2195–2204.

[28] J.M. Gonzalez-Barahona, P. Sherwood, G. Robles, D. Izquierdo, Technical lag in software compilations: Measuring how outdated a software deployment is, in: Open Source Systems: Towards Robust Practices, 2017, pp. 182–192.

[29] A. Decan, T. Mens, P. Grosjean, An empirical comparison of dependency network evolution in seven software packaging ecosystems, Empir. Softw. Eng. 24 (2018) 381–416.

[30] H. Kagdi, Mining Software Repositories to Support Software Evolution (Ph.D. thesis), Kent State University, USA, 2018.

[31] J. Kabbedijk, S. Jansen, Steering insight: An exploration of the ruby software ecosystem, in: International Conference on Software Business, ICSOB, Brussels, Belgium, 2011, pp. 44–55.

[32] N. Bezirgiannis, J. Jeuring, S. Leather, Usage of generic programming on hackage: Experience report, in: Proceedings of the SIGPLAN Workshop on Generic Programming, Boston, USA, 2013, pp. 47–52.

[33] D. German, B. Adams, A. Hassan, The evolution of the r software ecosystem, in: European Conference on Software Maintenance and Reengineering, IEEE, Genova, Italy, 2013.

[34] E. Wittern, P. Suter, S. Rajagopalan, A look at the dynamics of the JavaScript package ecosystem, in: International Conference on Mining Software Repositories, Austin, USA, 2016.

[35] A. Decan, T. Mens, M. Claes, On the topology of package dependency networks, in: Proccedings of the 10th European Conference on Software Architecture Workshops, Copenhagen, Denmark, 2016.

[36] A. Decan, T. Mens, M. Claes, P. Grosjean, When GitHub meets CRAN: An analysis of inter-repository package dependency problems, in: 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER, IEEE, Osaka, Japan, 2016, pp. 493–504, http://dx.doi.org/10.1109/saner.2016.12.

[37] A. Decan, T. Mens, M. Claes, An empirical comparison of dependency issues in OSS packaging ecosystems, in: 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, SANER, Klagenfurt, Austria, 2017, pp. 2–12.

[38] E. Bommarito, M. Bommarito, An empirical analysis of the python package index (PyPI), 2019, ArXiv - CoRR. http://arxiv.org/abs/1907.11073.

[39] B. Chinthanet, R.G. Kula, S. McIntosh, T. Ishio, A. Ihara, K. Matsumoto, Lags in the release, adoption, and propagation of NPM vulnerability fixes, Empir. Softw. Eng. 26 (2021) 47.

[40] A. Zerouali, T. Mens, A. Decan, C. De Roover, On the impact of security vulnerabilities in the NPM and RubyGems dependency networks, Empir. Softw. Eng. 27 (2022).

[41] J.D. Scarsbrook, M. Utting, R.K.L. Ko, TypeScript's evolution: An analysis of feature adoption over time, in: IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, Melbourne, Australia, 2023, pp. 109–114.

[42] X. Jia, Y. Zhou, Y. Hussain, W. Yang, An empirical study on python library dependency and conflict issues, in: IEEE 24th International Conference on Software Quality, Reliability and Security, QRS, Cambridge, United Kingdom, 2024, pp. 504–515.

[43] N. Vazou, A. Tondwal, P. Rondon, R. Jhala, Verifying replicated data types with typeclass refinements in liquid haskell, Proc. ACM Program. Lang. (OOPSLA Issue) 4 (2020) 216:1–216:30.

[44] Foundation Cardano, A public decentralized blockchain platform which uses the cryptocurrency, 2026, https://cardano.org. Last Visited: 21/01/2026.

[45] M.M.T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, J. Müller, M. Peyton Jones, P. Vinogradova, P. Wadler, Native custom tokens in the extended UTXO model, in: Leveraging Applications of Formal Methods, Verification and Validation: Applications, ISoLA 2020, in: Lecture Notes in Computer Science, vol. 12478, Springer, 2020, pp. 89–111.

[46] E. Moggi, Computational Lambda-Calculus and Monads, University of Edinburgh, Berlin, Heidelberg, 1988, pp. 42–122.

[47] P. Wadler, Comprehending monads, in: Proceedings of the 1990 ACM Conference on LISP and Functional Programming, Nice, France, 1990, pp. 61–78.

[48] S. Liang, P. Hudak, M. Jones, Monad transformers and modular interpreters, in: ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Francisco, USA, 1995, pp. 333–343.

[49] E. Moggi, Notions of computation and monads, Inform. and Comput. 93 (1991) 55–92, Selections from 1989 IEEE Symposium on Logic in Computer Science.

[50] W. De Meuter, Monads as a theoretical foundation for AOP, in: International Workshop on Aspect-Oriented Programming at ECOOP, Jyväskylä, Finland, 1997, pp. 31–36.

[51] T. Aotani, T. Kamina, H. Masuhara, Type-safe layer-introduced base functions with imperative layer activation, in: Proceedings of the 7th ACM International Workshop on Context-Oriented Programming, COP, Prague, Czech Republic, 2015, pp. 1–7.

[52] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Professional Computing Series, Addison-Wesley, 1994.

[53] M.P. Jones, Functional programming with overloading and higher-order polymorphism, in: Advanced Functional Programming, First International Spring School on Advanced Functional Programming Techniques-Tutorial Text, Båstad, Sweden, 1995, pp. 97–136.

[54] M. Algehed, A. Russo, Encoding DCC in haskell, in: Proceedings of the Workshop on Programming Languages and Analysis for Security, Dallas, USA, 2017, pp. 77–89.

[55] N. Benton, J. Hughes, E. Moggi, Monads and effects, in: G. Barthe, P. Dybjer, L. Pinto, J. Saraiva (Eds.), Applied Semantics, Caminha, Portugal, 2002, pp. 42–122.

[56] C. k. Lin, Programming monads operationally with unimo, in: Proceedings of International Conference on Functional Programming, ICFP, Portland, USA, 2006, pp. 274–285.

[57] P. Hudak, A. Courtney, H. Nilsson, J. Peterson, Arrows, robots, and functional reactive programming, in: Advanced Functional Programming, AFP, Oxford, UK, 2002, pp. 159–187.

[58] C. Elliott, P. Hudak, Functional reactive animation, in: Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming, ICFP'97, Amsterdam, The Netherlands, 1997, pp. 263–273.

[59] M. Jaskelioff, Monatron: An extensible monad transformer library, in: Lecture Notes in Computer Science, vol. 5836, 2011, pp. 233–248.

[60] A. Ross, V.L. Willson, One-way anova, Basic and Advanced Statistical Tests, Brill, 2017, pp. 21–24.

[61] H. Abdi, L.J. Williams, Newman-Keuls test and Tukey test, Encycl. Res. Des. 2 (2010) 897–902.

[62] Scrapy.org, An open source and collaborative framework for extracting the data you need from websites, 2024, https://www.scrapy.org. Last visited: 01/04/2024.

[63] Pandas, A fast, powerful, flexible and easy to use open source data analysis and manipulation tool, 2024, https://pandas.pydata.org. Last visited: 01/04/2024.

[64] Matplotlib, A library for creating static, animated, and interactive visualizations in python, 2024, http://matplotlib.org. Last visited: 01/04/2024.

[65] R. Tarjan, Depth-first search and linear graph algorithms, SIAM J. Comput. 1 (1972) 146–160.

[66] P. Leger, I. Figueroa, D. Burton, A. Bergmark, A. Lelechenko, A. Abel, Safety issues in stackage? 2022, https://github.com/commercialhaskell/stackage/issues/6590. Last Visited: 05/09/2023.

[67] S.A. Mirhosseini, C. Parnin, Can automated pull requests encourage software developers to upgrade out-of-date dependencies? Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering, IEEE, 2017, pp. 84–94.

[68] M. Vieira, D. Richardson, The role of dependencies in component-based systems evolution, in: International Workshop on Principles of Software Evolution, 2002, pp. 62–65.

[69] A. Serebrenik, T. Mens, Challenges in software ecosystems research, in: Proceedings of the 2015 European Conference on Software Architecture Workshops, 2015, pp. 1–6.

[70] S. Mujahid, R. Abdalkareem, E. Shihab, What are the characteristics of highly-selected packages? A case study on the npm ecosystem, J. Syst. Softw. 198 (2023) 111588.

[71] M.M. Lehman, Programs, life cycles, and laws of software evolution, Proc. IEEE 68 (1980) 1060–1076.

[72] I. Herraiz, D. Rodriguez, G. Robles, J.M. Gonzalez-Barahona, The evolution of the laws of software evolution: A discussion based on a systematic literature review, ACM Comput. Surv. 46 (2013) 1–28.

[73] P. Goswami, S. Gupta, Z. Li, N. Meng, D. Yao, Investigating the reproducibility of NPM packages, in: International Conference on Software Maintenance and Evolution, 2020, pp. 677–681.

[74] G.A.A. Prana, C. Bird, E.T. Barr, P.T. Devanbu, A. Hindle, Using practitioners' insights to investigate reproducibility of software engineering studies, Empir. Softw. Eng. 26 (2021) 1–37.

[75] N. Bezirgiannis, J. Jeuring, S. Leather, Usage of generic programming on hackage: Experience report, in: Proceedings of the 9th ACM SIGPLAN Workshop on Generic Programming, Boston, USA, 2013.